Python Decorator Reference

50 decorators across 9 groups

new new addition collapsed collapsed from list stdlib stdlib / built-in

Stdlib / language

`@property`	stdlib	Turn a method into an attribute accessor
`@classmethod`	stdlib	Bind method to class; receives cls not self
`@staticmethod`	stdlib	Namespace utility method; no self or cls
`@dataclass`	stdlib	Auto-generate __init__, __repr__, __eq__
`@abstractmethod`	stdlib	Force subclasses to implement the method
`@contextmanager`	stdlib	Turn a generator into a context manager
`@overload`	stdlib	Type-only polymorphic signatures (typing)
`@singledispatch`	stdlib	Runtime dispatch on first argument type
`@lru_cache(maxsize=N)`	stdlib	Stdlib memoization with bounded cache
`@functools.wraps`	stdlib	Preserve wrapped fn name/docs — use inside every decorator you write

Observability

`@log(level='INFO')`	collapsed	Log calls, args, results at configurable level — collapses @logger
`@timer(repeat=1, warn_threshold=None)`	collapsed	Elapsed time + optional statistical repeat + slow-call warning — collapses @timer / @benchmark / @warn_if_slow
`@trace`		Deep call-graph tracing; captures args and returns at every frame
`@audit`		Compliance-grade immutable record of sensitive actions
`@observe(service='X')`		Structured telemetry + distributed tracing (OpenTelemetry-style)
`@profile_cpu`		cProfile-based CPU profiling
`@monitor_memory`		Peak RAM tracking via tracemalloc

Reliability

`@retry(times=3, exceptions=..., backoff=1.0)`	collapsed	Retry with count, exception filter, backoff — collapses @retry / @retry(3)
`@timeout(seconds=30)`		Kill execution after N seconds
`@circuit_breaker(threshold=5)`	new	Fail fast after N consecutive failures; auto-reset after cooldown
`@fallback(fn)`		Switch to backup logic on failure
`@exception_handler`		Catch, log, and optionally re-raise exceptions

Performance & caching		
`@cache(ttl=None, maxsize=128)`	collapsed	ttl=None → pure memoize; ttl>0 → expiring cache — collapses @cache / @memoize
`@lazy_load`		Defer expensive init until first access
`@batch(size=32)`		Accumulate calls and dispatch together
`@vectorize`		Apply a scalar function element-wise over arrays
`@checkpoint(interval=100)`		Persist intermediate state every N steps

Concurrency & resources		
`@lock`		Prevent concurrent access (threading.Lock)
`@rate_limit(calls=100, period=60)`	collapsed	Cap calls per time window — collapses @rate_limit / @rate_limit(100)
`@throttle(min_interval=0.1)`	new	Enforce minimum gap between successive calls — distinct from rate_limit
`@background_task`		Dispatch to thread or async task
`@gpu_required(min_memory_gb=None)`		Assert GPU presence + optional VRAM floor

Validation & security	
`@validate(schema=None)`	Input validation; optional schema enforcement
`@sanitize`	Clean and escape inputs
`@authenticate`	Verify identity before execution
`@authorize`	Check permissions after authentication
`@require_role('admin')`	Role-based access control
`@guardrail`	AI safety / policy check on inputs or outputs

Data & state		
`@transaction`		Wrap function in a DB transaction (commit / rollback)
`@seed_randomness(seed=42)`		Fix random seeds for reproducibility
`@idempotent`	new	Assert or enforce that re-runs produce the same result

AI / ML	
`@tool('name')`	Register callable as an AI agent tool
`@prompt_template`	Auto-format prompt strings before the call
`@schema_enforcer`	Enforce structured output from an LLM response
`@inject_dependency`	Auto-inject dependencies — framework-specific (e.g. FastAPI Depends)

Lifecycle / meta	
`@deprecated(version='X')`	Warn on use; document the replacement
`@feature_flag('flag_name')`	Toggle feature at runtime without deploy
`@register_plugin`	Auto-register in plugin registry on import
`@singleton`	One instance only — note: decorates a class, not a function
`@track_usage`	Record call metrics and analytics